

Objetivos Plan de Calidad

Nombre del proyecto: Orion

Nombre del producto: ORION



“El guía que ilumina tu futuro”

Daniel Feliz Aguilar Rodriguez

Camilo Andrés Aragón Román

Juan David Montealegre Guzmán

Sergio Alejandro Pedraza Rincon

Evaluación de requisitos

Nombre	Características	Función	Como se evalúa
--------	-----------------	---------	----------------

Comparador de Pénsums	Tabla/Matriz dinámica Selección de dos carreras Visualización paralela de mallas Tabla o matriz dinámica Resaltado de similitudes Identificación de prerequisites	El sistema debe permitir seleccionar dos carreras y visualizar sus mallas curriculares simultáneamente en una vista comparativa. Las materias equivalentes o similares deben destacarse visualmente para facilitar la comparación.	Se verifica que el sistema permita seleccionar exactamente dos carreras desde la interfaz. Se comprueba que ambas mallas curriculares se visualicen de forma simultánea en una vista comparativa. Se valida que las materias equivalentes o similares estén resaltadas visualmente (color, ícono o etiqueta). Se revisa que los prerequisites estén claramente identificados en cada malla. Pruebas funcionales con diferentes combinaciones de carreras para confirmar consistencia.
Gestión de Ficha Técnica	Formulario de edición Operaciones CRUD (crear, leer, editar, eliminar) Panel administrativo Edición de costos y duración Actualización inmediata de datos Restricción por rol	Los usuarios con rol de administrador universitario deben poder crear, modificar y actualizar la información de las carreras, incluyendo costos, duración, modalidad y enlaces oficiales. Los cambios deben almacenarse inmediatamente en la base de datos.	Se accede al sistema con un usuario con rol de administrador y se comprueba el acceso al panel administrativo. Se realizan operaciones CRUD completas (crear, leer, editar y eliminar carreras). Se valida que los cambios en costos, duración, modalidad y enlaces se reflejen de manera inmediata. Se consulta la base de datos para confirmar la

			persistencia de la información. Se prueba con un usuario sin permisos para confirmar la restricción por rol.
Cameo de Extensión	Inyección de DOM Extensión de navegador Detección automática de URL Inyección de contenido DOM Ventana emergente informativa Visualización no intrusiva	La extensión del navegador debe detectar cuando el usuario visite un sitio web universitario registrado y mostrar automáticamente un resumen informativo con datos relevantes de la carrera. El contenido debe desplegarse como ventana emergente sin interrumpir la navegación.	Se instala la extensión en el navegador y se verifica su correcto funcionamiento. Se accede a un sitio web universitario registrado y se valida la detección automática de la URL. Se comprueba que el contenido informativo se inyecte correctamente en el DOM. Se valida que la ventana emergente sea no intrusiva y no interrumpa la navegación. Pruebas de compatibilidad en distintos sitios y navegadores.
Filtro de búsqueda	Barra de búsqueda Búsqueda por múltiples criterios Filtros combinables Resultados dinámicos Actualización en tiempo real Interfaz interactiva	El sistema debe permitir filtrar carreras mediante criterios como ciudad, precio, modalidad y universidad. Los resultados deben actualizarse dinámicamente según los filtros seleccionados.	Se prueba la barra de búsqueda ingresando distintos criterios (ciudad, precio, modalidad, universidad). Se verifica que los filtros puedan combinarse correctamente. Se comprueba que los resultados se

			actualicen en tiempo real sin recargar la página. Se valida la precisión y relevancia de los resultados mostrados. Pruebas de usabilidad para asegurar una interfaz clara e interactiva.
Cierre de sesión	Control de sesión Botón visible de salida Finalización de sesión activa Eliminación de credenciales temporales Redirección a inicio Protección contra acceso posterior	El sistema debe permitir al usuario cerrar sesión manualmente en cualquier momento. Al cerrar sesión, se deben invalidar las credenciales activas y redirigir al usuario a la pantalla de inicio.	Se verifica la existencia y visibilidad del botón de cerrar sesión. Se comprueba que al cerrar sesión se invalide la sesión activa. Se valida la eliminación de credenciales temporales (tokens o sesiones). Se confirma la redirección automática a la pantalla de inicio. Se intenta acceder a secciones protegidas después del cierre para comprobar la protección contra acceso no autorizado.
Registro de aspirantes	Formulario digital de registro Creación de cuenta de usuario Campos obligatorios Validación de correo único Mensajes de confirmación y error	El sistema debe permitir a los aspirantes crear una cuenta ingresando nombre completo, número de documento, correo electrónico y contraseña. Todos los campos son obligatorios. El correo electrónico debe ser único en la base de datos. El sistema debe	Se realiza un registro con toda la información necesaria. Se debe verificar que no haya errores en el momento de registro. Se verifica en la base de datos que no se repita el correo electrónico en ningún registro más, y se verifica toda la información que se

		mostrar mensajes de confirmación o error según el resultado del registro.	haya registrado en la prueba sea correcta.
Validación de datos	Validación automática de formato Control de entradas inválidas Mensajes de error en tiempo real Retroalimentación visual inmediata Prevención de envío con datos incorrectos	El sistema debe validar automáticamente el formato del documento y del correo electrónico antes de permitir el envío del formulario. En caso de error, debe mostrar mensajes descriptivos junto al campo correspondiente sin recargar la página.	Se deben probar diversas variaciones de correos y documentos para comprobar que no se repita en ningún momento y tenga validación con la base de datos. Verificar los mensajes emergentes poniendo datos erróneos verificando su correcta función.
Procesamiento de Prompts	Interfaz de IA (NLP) Entrada de texto en lenguaje natural Análisis semántico de consultas Motor de recomendaciones Filtrado dinámico de carreras y universidades Registro de consultas para analítica	El sistema debe permitir al aspirante ingresar consultas en lenguaje natural para obtener recomendaciones de carreras y universidades. El sistema debe analizar el texto ingresado y filtrar la información académica disponible mostrando resultados relevantes.	Realizar diversas consultas para obtener diferentes recomendaciones variando sus respuestas. Verificar la información propuesta por el sistema y sus resultados.

Autenticación de usuario	Verificación de credenciales Inicio de sesión con credenciales Verificación de correo y contraseña Gestión de sesiones activas Bloqueo tras intentos fallidos Redirección al dashboard	El sistema debe permitir el inicio de sesión mediante correo electrónico y contraseña registrados. Las credenciales deben validarse antes de conceder acceso al dashboard. Después de tres intentos fallidos consecutivos, la cuenta debe bloquearse temporalmente por motivos de seguridad.	Intentar variaciones de correo y contraseña para verificar el correcto registro individual de cada correo electrónico y sus contraseñas establecidas. Verificar el estado de bloqueo y su correcto funcionamiento al no poderlo introducir durante un tiempo establecido.
Recuperación de contraseña	Proceso de recuperación Envío de correo automático Generación de token temporal Restablecimiento seguro de contraseña Tiempo de expiración del enlace Confirmación de cambio exitoso	El sistema debe permitir al usuario recuperar su contraseña mediante el envío de un enlace o token temporal al correo electrónico registrado. El token debe tener tiempo de expiración y permitir establecer una nueva contraseña segura.	Intentar varios cambios de contraseña para verificar su correcto envío y que este no se pueda realizar múltiples veces en un periodo de tiempo. Asegurar el token temporal sea funcional solo para ese individuo durante el tiempo establecido. Verificar la base de datos a los cambios de contraseñas realizados.

REQUERIMIENTOS NO FUNCIONALES

Nombre	Características	Función	Como se evalúa
--------	-----------------	---------	----------------

Seguridad en contraseñas	• Validación robusta • Longitud mínima obligatoria • Complejidad alfanumérica • Validación automática • Protección contra contraseñas débiles	Las contraseñas deben tener mínimo 8 caracteres e incluir letras mayúsculas, números y al menos un carácter especial.	Intentar crear contraseñas que no cumplan con los requisitos y verificar que el sistema los rechace.
Disponibilidad del servicio	• Acceso permanente • Acceso continuo 24/7 • Alta disponibilidad • Tolerancia a fallos • Mínimo tiempo de inactividad	El sistema debe mantener disponibilidad continua del 99% o superior, garantizando acceso permanente al registro, autenticación y buscador.	Usar herramientas como UptimeRobot o Pingdom para medir tiempos de actividad, y simular fallos de servidor para medir tiempos de recuperación.
Tiempo de respuesta	• Validación rápida • Baja latencia • Procesamiento rápido • Respuesta menor a 1 segundo • Optimización de consultas	Las operaciones de autenticación y procesamiento de consultas deben completarse en un tiempo máximo de 2 segundos bajo condiciones normales de uso.	Revisando que los tiempos de respuesta deban estar en < 1s y simular una alta concurrencia para medir la latencia promedio.
Usabilidad de la interfaz	• Interfaz clara e intuitiva • Diseño intuitivo • Navegación simple • Consistencia visual • Aprendizaje rápido • Acciones con pocos pasos	El dashboard debe ser intuitivo, consistente y fácil de utilizar por usuarios sin conocimientos técnicos, minimizando la cantidad de pasos para completar tareas principales., destacando el Rojo Orion .	Pruebas con usuarios reales, al seleccionarlos para hacer uso de la página y reevaluar las preguntas que tengan del funcionamiento para hacer el sistema más claro.

Escalabilidad del sistema	• Manejo de carga • Soporte a múltiples usuarios concurrentes • Crecimiento horizontal • Balanceo de carga • Rendimiento estable bajo demanda	El backend en Python El sistema debe soportar múltiples consultas simultáneas sin degradación significativa del rendimiento, permitiendo el incremento de usuarios mediante escalamiento horizontal.	Al plantear escenarios de alta latencia verificar que, al añadir nodos, el tiempo de respuesta se mantenga estable.
Seguridad de datos académicos	• Confidencialidad y encriptación • Cifrado de información • Control de acceso • Protección contra accesos no autorizados • Resguardo de datos personales	Toda la información académica y personal debe almacenarse utilizando mecanismos de cifrado y controles de acceso para evitar accesos no autorizados.	Realizar pruebas de control de acceso al intentar acceder a datos de otros usuarios sin permiso, debiendo estos fallar.
RNF07 – Integridad de la información	- Datos consistentes- Consistencia de datos- Prevención de pérdida- Respaldo periódico- Validación de transacciones	El sistema debe garantizar que los datos académicos y curriculares no se pierdan ni se modifiquen incorrectamente durante operaciones de actualización o consulta.	Se verifica que las operaciones de actualización o consulta no generen pérdida ni modificación incorrecta de datos.
RNF08 – Control de roles	- Acceso restringido- Autorización basada en permisos- Restricción por perfil- Separación de privilegios- Gestión segura de accesos	El acceso a funcionalidades administrativas debe estar restringido según el rol del usuario, permitiendo únicamente a personal autorizado modificar información institucional.	Se comprueba que los usuarios solo accedan a funciones permitidas según su rol y que las restricciones de permisos sean efectivas.
RNF09 – Rendimiento del Dashboard	- Carga dinámica rápida- Carga rápida de vistas- Renderizado eficiente-	Las vistas comparativas y paneles de	Se mide el tiempo de carga de las vistas y se valida

	Optimización de consultas- Tiempo máximo de 3 segundos	pésums deben cargarse completamente en un tiempo máximo de 3 segundos.	que no supere los 3 segundos en condiciones normales.
RNF10 – Accesibilidad Multiplataforma	- Compatibilidad total- Diseño responsive- Compatibilidad con navegadores modernos- Adaptación móvil- Interfaz consistente en diferentes pantallas	La plataforma y la extensión deben ser compatibles con navegadores modernos y adaptarse correctamente a dispositivos móviles, tabletas y computadores.	Se verifica el correcto funcionamiento y visualización en navegadores y dispositivos distintos, comprobando la adaptabilidad y consistencia visual.
RNF11 – Consistencia de datos	- Actualización inmediata- Sincronización automática- Información en tiempo real- Evitar datos desactualizados	Los cambios realizados en la información académica deben reflejarse en el sistema en un tiempo máximo de 5 segundos para todos los usuarios.	Se verifica que los cambios efectuados en la información sean visibles para todos los usuarios dentro de los 5 segundos siguientes.
RNF12 – Accesibilidad	- Compatibilidad- Cumplimiento básico WCAG- Alto contraste visual- Navegación por teclado- Texto legible- Soporte a usuarios con limitaciones visuales	La interfaz debe cumplir estándares básicos de accesibilidad web, garantizando correcta visualización, contraste y navegación mediante teclado.	Se valida el cumplimiento de las pautas WCAG mediante pruebas de contraste, navegación por teclado y legibilidad del texto.

Historia de usuario.

Historia de usuario	
Número: 01	Usuario: Aspirante
Nombre Historia: Consulta de orientación por Prompt (IA)	

Prioridad en negocio: Alta	Riesgo en desarrollo: Alto
Puntos estimados: 8	Puntos reales:
Programador Responsable: Backend	
Descripción: Como aspirante, quiero ingresar mis gustos y presupuesto en lenguaje natural para que el sistema me recomiende las mejores opciones de carrera.	
Observaciones: El sistema debe procesar el texto mediante lenguaje natural y filtrar la base de datos de MongoDB.	

Historia de usuario	
Número: 02	Usuario: Aspirante
Nombre Historia: Comparación interactiva de Pénsums	
Prioridad en negocio: Alta	Riesgo en desarrollo: Medio
Puntos estimados: 5	Puntos reales:
Programador Responsable: Frontend	
Descripción: Como aspirante, quiero seleccionar dos carreras para ver sus mallas curriculares lado a lado y entender sus diferencias de materias.	
Observaciones : Utilizar componentes visuales (Cards) y resaltar materias similares con colores.	

Historia de usuario	
Número: 03	Usuario: Universidad
Nombre Historia: Gestión de "Ficha Técnica" de carrera	
Prioridad en negocio: Alta	Riesgo en desarrollo: Bajo
Puntos estimados: 5	Puntos reales:

Programador Responsable: Backend	
Descripción: Como administrador universitario, quiero actualizar costos, duración y links oficiales de las carreras para que los aspirantes vean información real.	
Observaciones: Debe incluir validación de campos numéricos para los precios y formatos de URL.	

Historia de usuario	
Número: 04	Usuario: Médico Universidad
Nombre Historia: Visualización de analíticas de interés	
Prioridad en negocio: Media	Riesgo en desarrollo: Medio
Puntos estimados: 6	Puntos reales:
Programador Responsable: Backend	
Descripción: Como universidad, quiero ver cuáles de mis carreras son las más consultadas mediante los prompts de los usuarios para ajustar mi oferta.	
Observaciones: El sistema debe registrar cada interacción exitosa del prompt sin comprometer la privacidad del usuario.	

Historia de usuario	
Número: 05	Usuario: Aspirante
Nombre Historia: Activación de "Cameo" Informativo	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alto
Puntos estimados: 8	Puntos reales:
Programador Responsable: Frontend	

Descripción: Como usuario de la extensión, quiero que al navegar en la web de una universidad aparezca un resumen inteligente (costo, acreditación) de la carrera que estoy viendo.
Observaciones: Requiere inyección de scripts en el DOM y reconocimiento de URLs registradas en MongoDB.

Objetivos del plan de pruebas

Objetivo general:

El objetivo principal del plan de pruebas es verificar el correcto funcionamiento del sistema que implementa estructuras de grafos, asegurando que las operaciones y representaciones de grafos dirigidos y no dirigidos se ejecuten de manera adecuada. A través de estas pruebas se busca validar que los nodos y las aristas se registren correctamente, que las relaciones entre los vértices correspondan a la lógica definida y que el sistema represente de forma precisa la dirección o ausencia de dirección en las conexiones, tal como se observa en los ejemplos de grafos.

Objetivos específicos:

1. Uno de los objetivos es comprobar que el sistema identifique correctamente los vértices y las aristas dentro del grafo, verificando que cada nodo pueda conectarse con otros según las reglas establecidas. Esto implica validar que las conexiones entre los nodos se almacenen y se visualicen correctamente dentro de la estructura de datos utilizada por el programa.
2. objetivo es verificar el manejo adecuado de grafos dirigidos, asegurando que las aristas mantengan su dirección correcta, es decir, que la relación entre dos nodos se interprete como un camino que va desde un vértice origen hacia un

vértice destino. De esta forma se garantiza que las operaciones que dependen de la dirección de las aristas funcionen correctamente.

3. **validar el comportamiento de los grafos no dirigidos**, donde las conexiones entre los nodos no poseen orientación específica. En este caso, las pruebas deben confirmar que el sistema interprete las aristas como relaciones bidireccionales entre los vértices.
4. **detectar posibles errores en la creación, almacenamiento y visualización del grafo**, permitiendo identificar fallos en la lógica del sistema antes de su implementación final. Con esto se garantiza que la representación y manipulación de los grafos sea confiable, consistente y adecuada para su uso dentro de la aplicación.

Herramientas por utilizar para las pruebas

1. Capa de Servidor (Backend)

La lógica de negocio y el motor de inferencia existen en el *backend*. Para validar la exactitud de los algoritmos de recomendación, se proponen las siguientes herramientas:

- **Pytest:** Seleccionado como el *framework* principal para el desarrollo de pruebas unitarias. Su arquitectura permite una gestión modular mediante *fixtures*, facilitando la simulación de estados complejos en la base de datos sin comprometer la persistencia de los datos reales.
- **HTTPX:** Implementado para la ejecución de pruebas de integración sobre los *endpoints* de la API. Esta herramienta permite validar el flujo de entrada/salida (*request/response*) bajo condiciones controladas sin necesidad de instanciar el servidor en un entorno productivo.
- **Coverage.py:** Herramienta esencial para la métrica de cobertura de código (*code coverage*), permitiendo identificar segmentos críticos de la lógica de decisión que no han sido validados.

2. Capa de Cliente (Frontend)

El *frontend* actúa como el punto de interacción con el usuario. Para asegurar que la lógica de presentación y el estado de la aplicación sean consistentes, se utilizará:

- **Vitest:** *Test runner* nativo para entornos Vite. Destaca por su alta velocidad de ejecución y compatibilidad total con TypeScript, lo cual es fundamental para verificar la coherencia de las interfaces definidas en `types.ts`.
- **React Testing Library:** Enfocada en la validación de la experiencia del usuario (UX) mediante el testeo de componentes funcionales, asegurando que las respuestas del formulario se procesen y reflejen correctamente en la interfaz.
- **Mock Service Worker (MSW):** Herramienta técnica de intercepción de tráfico HTTP. Su uso es crítico para simular las respuestas del *backend* durante las pruebas de interfaz, permitiendo desacoplar el desarrollo del *frontend* de la disponibilidad del servidor.

3. Integración Continua y Calidad del Código

Para garantizar la mantenibilidad del proyecto, se recomienda la adopción de herramientas de análisis estático:

- **ESLint y Prettier:** Implementados para la normalización del estilo y la detección temprana de errores de sintaxis o patrones de código anti-patrones en el entorno TypeScript.

Clases de prueba

Gestión de acceso

- Verificar acceso al sistema con usuario administrador.
- Verificar que usuarios sin permisos no accedan al panel administrativo.

Gestión de carreras (CRUD)

- Crear una carrera.
- Leer/consultar información de una carrera.
- Editar datos de una carrera.
- Eliminar una carrera.

Validación de datos

- Validar edición de costos.
- Validar duración de la carrera.
- Validar modalidad.
- Validar enlaces oficiales.

Persistencia de datos

- Confirmar que los cambios se guardan en la base de datos.
- Verificar actualización inmediata de la información.
- Restricción por roles
- Usuario administrador puede modificar datos.
- Usuario normal no puede crear, editar ni eliminar carreras.

Tipos de prueba

Pruebas funcionales

- Verificar que las operaciones CRUD funcionen correctamente.

Pruebas de seguridad

- Validar que solo el administrador pueda modificar la información.

Pruebas de integración

- Confirmar que los cambios en el panel se reflejen en la base de datos.
- Pruebas de validación de datos
- Comprobar que los campos acepten solo datos válidos.
- Pruebas de consistencia
- Probar diferentes combinaciones de carreras para confirmar que el sistema mantenga la información correcta.

Cronograma o proyección de tiempos para la ejecución de las pruebas

El siguiente cronograma presenta la **planificación estimada para la ejecución del proceso de pruebas del sistema Orion**, considerando las distintas fases del ciclo de aseguramiento de la calidad, los responsables involucrados y el tiempo requerido para cada actividad. Este cronograma permite organizar los esfuerzos del equipo, asegurar la cobertura de los requisitos funcionales y no funcionales, y facilitar el seguimiento del avance del plan de pruebas.

Actividad	Responsable	Duración estimada	Observaciones y herramientas utilizadas
Planificación de pruebas	Líder QA	3 días	Definición del alcance, requisitos a validar y diseño de casos de prueba. Uso de documentación funcional, historias de usuario y criterios de calidad.
Preparación del entorno	DevOps / QA	2 días	Configuración de ambientes de prueba, despliegue del backend y frontend, carga de datos controlados en MongoDB y configuración de variables de entorno.
Ejecución de pruebas unitarias	Equipo de desarrollo	3 días	Validación de componentes individuales. Backend probado con Pytest y Coverage.py ; frontend validado con Vitest y pruebas de componentes con React Testing Library .
Pruebas de integración	QA / Desarrolladores	3 días	Verificación de la comunicación entre API y frontend mediante HTTPX para endpoints y Mock Service Worker (MSW) para simulación de respuestas del backend.
Pruebas funcionales	Equipo QA	4 días	Validación del cumplimiento de los requisitos funcionales desde la perspectiva del usuario final, utilizando escenarios reales y pruebas manuales guiadas por casos de uso.
Pruebas no funcionales	QA / Seguridad	3 días	Evaluación de rendimiento, seguridad, accesibilidad y escalabilidad. Uso de métricas de tiempo de respuesta, validaciones de contraseñas,

			control de roles y pruebas de carga.
Pruebas de aceptación de usuario (UAT)	QA / Usuarios clave	3 días	Validación del sistema con usuarios finales para confirmar que las funcionalidades cumplen las expectativas del negocio y los objetivos del proyecto.
Corrección de defectos y re-ejecución	Desarrolladores / QA	3 días	Corrección de errores detectados y re-ejecución de pruebas unitarias, de integración y funcionales para verificar la solución de los defectos.
Cierre y reporte	Líder QA	2 días	Consolidación de resultados, métricas de calidad, reporte de cobertura de pruebas y cierre formal del ciclo de pruebas.